CS336 Assignment 2: Systems — Writeup

James Liu

May 2, 2026

1 Profiling and Benchmarking

Problem (`benchmarking_script`) *4 points*

(a) Write a script to perform basic end-to-end benchmarking of the forward pass, backward pass, and optimizer step in your model (given hyperparameters, random batch, warm-up steps, timed steps for forward / forward+backward / full training step, with `torch.cuda.synchronize()` after each step). *Deliverable:* A script that initializes a basics Transformer with given hyperparameters, creates a random batch, and times forward-only, forward-and-backward, and full training steps.

Answer.

See *cs336_systems/benchmarking.py*

(b) Time the forward, backward, and optimizer step for each model size in Table 1 using 5 warmup steps and 10 measurement steps. How long does a forward pass take? A backward pass? Is variability / std small? *Deliverable:* A 1–2 sentence response with your timings.

Answer.

Table 1: Benchmark results across model sizes (5 warm up steps, 10 training steps).

Size	forward	forward-back	forward-back-optimizer
small	0.0165 ± 0.0000	0.0488 ± 0.0000	0.0561 ± 0.0000
med	0.0472 ± 0.0001	0.1398 ± 0.0007	0.1566 ± 0.0011
large	0.1063 ± 0.0000	0.3168 ± 0.0069	0.3447 ± 0.0009
xl	0.2935 ± 0.0004	0.8636 ± 0.0005	0.9453 ± 0.0001
10b	0.9448 ± 0.0002	2.8113 ± 0.0006	OOM

See table for timing of forward, forward-backward, and forward-backward-optimizer steps. The forward-only, forward+backward, and full training-step timings are shown in Table 1. Forward+backward is roughly 3x slower than forward-only due to gradient computation, and adding the optimizer step introduces a smaller additional cost. The standard deviations are extremely small after warmup, which shows that the GPU runs are extremely consistent.

(c) Repeat the analysis without warmup, and with 1 or 2 warmup steps. How do results change? Why? *Deliverable:* A 2–3 sentence response.

Answer.

Table 2: Benchmark results across model sizes (no warm up steps, 10 training steps).

Size	forward	forward-back	forward-back-optimizer
small	0.0555 ± 0.1168	0.1181 ± 0.2001	0.1161 ± 0.1653
med	0.0817 ± 0.1043	0.2044 ± 0.1923	0.2330 ± 0.2127
large	0.1070 ± 0.0024	0.3605 ± 0.1424	0.3610 ± 0.0468
xl	0.3300 ± 0.1033	0.8694 ± 0.0265	1.0079 ± 0.1902
10b	0.9624 ± 0.0501	2.8153 ± 0.0015	OOM

As seen above in the table, without warming up the GPU beforehand, each of the computations take significantly longer (since now there is the overhead for warming up the GPU–compiling the kernels, allocating memory, etc...)

Table 3: Benchmark results across model sizes (one warm up step and 10 training steps).

Size	forward	forward-back	forward-back-optimizer
small	0.0164 ± 0.0000	0.0483 ± 0.0002	0.0634 ± 0.0187
med	0.0469 ± 0.0001	0.1390 ± 0.0000	0.1586 ± 0.0032
large	0.1062 ± 0.0001	0.3135 ± 0.0001	0.3496 ± 0.0222
xl	0.2927 ± 0.0003	0.8640 ± 0.0005	0.9422 ± 0.0004
10b	0.9451 ± 0.0008	2.8198 ± 0.0001	OOM

Table 4: Benchmark results across model sizes (2 warm up steps and 10 training steps).

Size	forward	forward-back	forward-back-optimizer
small	0.0165 ± 0.0000	0.0489 ± 0.0000	0.0569 ± 0.0006
med	0.0471 ± 0.0000	0.1396 ± 0.0001	0.1573 ± 0.0003
large	0.1064 ± 0.0005	0.3185 ± 0.0105	0.3443 ± 0.0007
xl	0.2927 ± 0.0003	0.8641 ± 0.0006	0.9453 ± 0.0009
10b	0.9452 ± 0.0003	2.8204 ± 0.0010	OOM

As seen in the above table comapred to the one without warm up steps, just warming up the GPU for one iteration greatly reduces the total training time. The result between 1 or 2 warm up steps is much closer to the 10 warm up steps one since most of the overhead (e.g. CUDA context initialization, allocator warm up, etc...) is eliminated if we just do one warm up step ahead of timing the GPU. The difference was also quite dramatic, showing that the majority of the time went into initializing the GPU and not running the forward and backward steps.

Problem (`nsys_profile`) *5 points*

Profile the forward pass, backward pass, and optimizer step with `nsys` for two model sizes and three power-of-two context lengths larger than 128.

Context Length	**Model Size**	
	Small (ms)	XL (ms)
256	17.794	77.585
512	30.700	112.659
1024	37.988	221.650

Table 5: Forward pass latency (ms) across model sizes and context lengths.

(a) Total forward-pass time in Nsight vs. the Python standard library time. *Deliverable:* A 1–2 sentence response.

Answer.

The Python standard-library timings closely matched the Nsight forward-pass timings after converting units. For the small model, Python measured 18.6 ms, 31.8 ms, and

39.5 ms at context lengths 256, 512, and 1024, compared with Nsight's 17.8 ms, 30.7 ms, and 38.0 ms. For the XL model, Python measured 80.4 ms, 116.9 ms, and 229.7 ms, compared with Nsight's 77.6 ms, 112.7 ms, and 221.7 ms.

(b) Which CUDA kernel has the largest cumulative GPU time in the forward pass? How many times is it invoked in one forward pass? Is it still the top kernel for forward+backward? *Deliverable:* A 1–2 sentence response.

Answer.

The dominant forward-pass CUDA kernel was a CUTLASS SGEMM matrix multiplication. For the XL model at context length 1024, the top kernel used a $128 \times 64 \times 16$ SGEMM tile, took 42.1% of GPU kernel time, and ran about 160 times per forward pass. In forward+backward, the dominant kernel was still SGEMM, but it switched to backward-shaped transpose variants.

(c) What non-matmul kernels account for non-trivial CUDA runtime in the forward pass? *Deliverable:* A 1–2 sentence response.

Answer.

Besides matrix multiplications, non-trivial forward-pass CUDA time came from elementwise kernels such as division, `where`, add, multiply, exponentiation, sigmoid, copy-/concat kernels, and reduction kernels used for max, sum, and normalization. For the XL model at context length 1024, matrix multiplications accounted for 86.8% of GPU kernel time, leaving about 13.2% for these non-matmul kernels.

(d) Profile a complete training step (forward, loss, backward, AdamW step). How does the matmul fraction change vs. inference? What about other kernels? *Deliverable:* A 1–2 sentence response.

Answer.

For the small model at context length 1024, the matmul fraction dropped from 69.6% in forward-only to 64.7% in the full forward+backward+optimizer step. For the XL model, it dropped from 86.8% to 82.3%. This happens because backward and AdamW introduce many additional pointwise/vectorized kernels, so non-matmul work becomes a larger fraction even though GEMMs still dominate.

(e) Compare softmax runtime to attention matmul runtime in the forward pass. How does the runtime ratio compare to the FLOP ratio? *Deliverable:* A 1–2 sentence response.

Answer.

The softmax operation did not appear as one fused softmax kernel; it was decomposed into max reductions, exponentials, sum reductions, and division. For the XL model at context length 1024, these softmax-like kernels were roughly 4–5% of forward GPU kernel time, while matmuls were 86.8%. Thus matmul runtime was about an order of magnitude larger, much less extreme than the FLOP gap, because softmax is memory-, reduction-, and special-function-heavy and runs less efficiently than Tensor Core GEMMs.

2 Mixed Precision

Problem (`mixed_precision_accumulation`) *1 points*

Run the accumulation snippet with FP32 / FP16 / mixed accumulators and comment on the accuracy. *Deliverable:* A 2–3 sentence response.

Answer.

The resulting answer to the code is below:

tensor(10.0001) tensor(9.9531, dtype=torch.float16) tensor(10.0021) tensor(10.0021)

All of the answers were off, but the float16 was off the most and the errors went down as you increased the precision. This is because 0.01 is not representable exactly in base-2, so there were slight rounding errors with each addition, and the cumulative error was the most for the float16 answers, which had the most rounding error per operation.

Problem (`benchmarking_mixed_precision`) *2 points*

(a) For the ToyModel under FP16 autocast, give the dtypes of: model parameters (inside autocast), output of `fc1`, output of `ln`, logits, loss, and gradients. *Deliverable:* The data types for each of the components listed above.

Answer.

- Model parameters (in autocast): model parameters remain in FP32 for storage
- Output of `fc1`: The linear layers are done in FP16 under autocast, so the output would be FP16.
- Output of `ln`: FP32
- Predicted logits: FP16
- Loss: FP32
- Gradients: FP32

(b) What parts of layer normalization are sensitive to mixed precision? Does BF16 vs FP16 change whether you need to treat LN differently? Why? *Deliverable:* A 2–3 sentence response.

Answer.

The mean/variance computation part of the layernorm is sensitive to mixed precision (subtracting the mean, squaring, summing, divison) since they can lose accuracy under FP16, so you have to make sure they are in FP32. BF16 has a wider dynamic range, giving it more training stability for layer normalization, so you probably don't need to cast it into FP32, but it has low mantissa precision, so accuracy may be worse.

(c) Benchmark full precision vs BF16 mixed precision for each model size, noting trends with model size. *Deliverable:* A 2–3 sentence response with timings and commentary.

Answer.

Table 6: Benchmark results across model sizes with mixed precision.

Size	forward	forward-back
small	0.0156 ± 0.0000	0.0475 ± 0.0018
med	0.0396 ± 0.0000	0.1145 ± 0.0001
large	0.0730 ± 0.0001	0.2180 ± 0.0132
xl	0.1113 ± 0.0000	0.3475 ± 0.0139
10b	0.2519 ± 0.0005	0.8117 ± 0.0069

(See question 1b for comparison full precision table)

The mixed precision runs ran faster than the full precision runs in general. The differences in run time was more dramatic as we increased model size, saving us much more time.

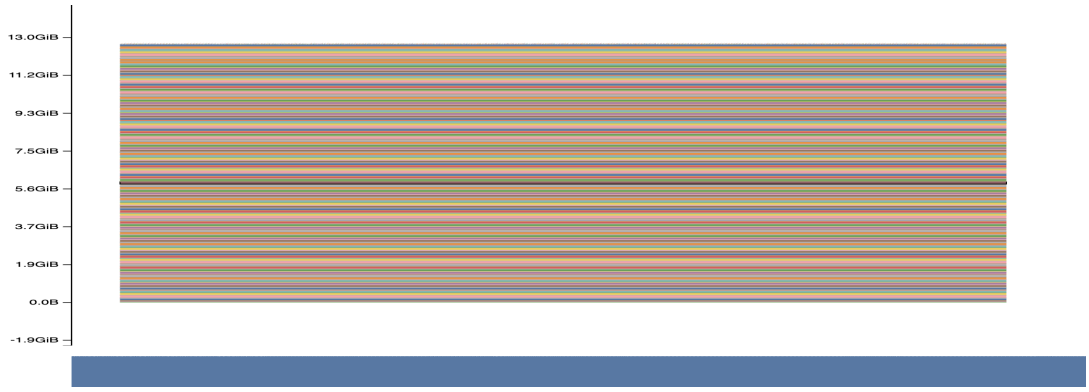
3 Memory Profiling

Problem (`memory_profiling`) *4 points*

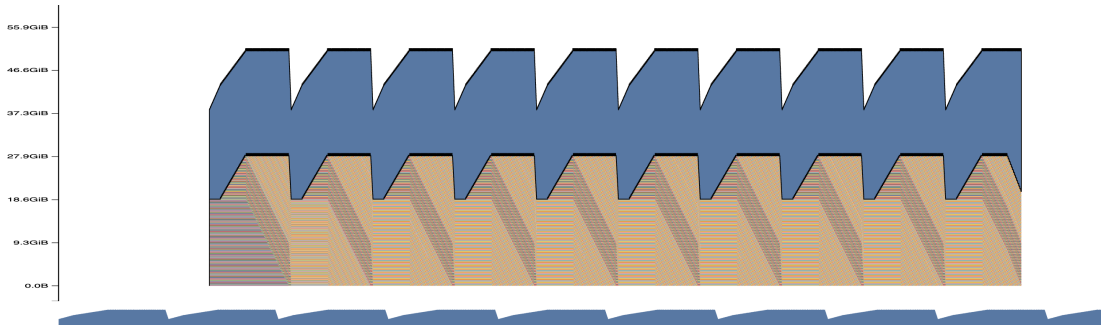
Profile the xl model with context lengths 128 and 2048 (forward-only and full training step).

(a) Show the "Active memory timeline" for a forward pass and a full training step. Can you tell which stage is which from the peaks? *Deliverable:* Two `memory_viz` screenshots (forward-only, full training step) and a 2–3 sentence response.

Answer.



Active memory timeline for the XL forward-only pass at context length 128.



Active memory timeline for the XL full training step at context length 128.

For the forward pass, my memory timeline looks flat, which makes sense since the forward pass doesn't include any memory intensive activites (when running @*torch.no_grad*() as well).

On the other hand, the full training pipeline has memory graph of repeated saw-tooth like patterns, this is probably from the memory accumulation during forward (saving activations), then you see a sudden drop after loss.backward() is ran and the activations are freed after gradient computation.

(b) Peak memory for each context length, forward pass vs. full training step. *Deliverable:* A table with two numbers per context length.

Answer.

Context length	Forward-only peak (MiB)	Full-step peak (MiB)
128	12.8GiB	51.2GiB
2048	21.2GiB	OOM

(c) Peak memory with mixed precision (forward-only and full step). Does MP significantly affect memory? *Deliverable:* A 2–3 sentence response.

Answer.

2048 context length with mixed precision for just a forward step has a peak memory of 25.4GiB, the 128 context length with mixed precision has 19GiB at peak, and the 128 full step has 51.2GiB at peak. Mixed precision does not significantly reduce memory usage. I suspect this is because of the overhead that torch.autocast brings.

(d) Size of a residual-stream activation tensor for the xl config in FP32, in MiB. *Deliverable:* A 1–2 sentence response with your derivation.

Answer.

In single precision (FP32), for model context length of 128, batch size of 4, embedding dimension of 2560, and at 4 bytes per number (FP32), the size of a tensor of activations would be 5,242,880 bytes. $5,242,880/1024^2$= 5MiB

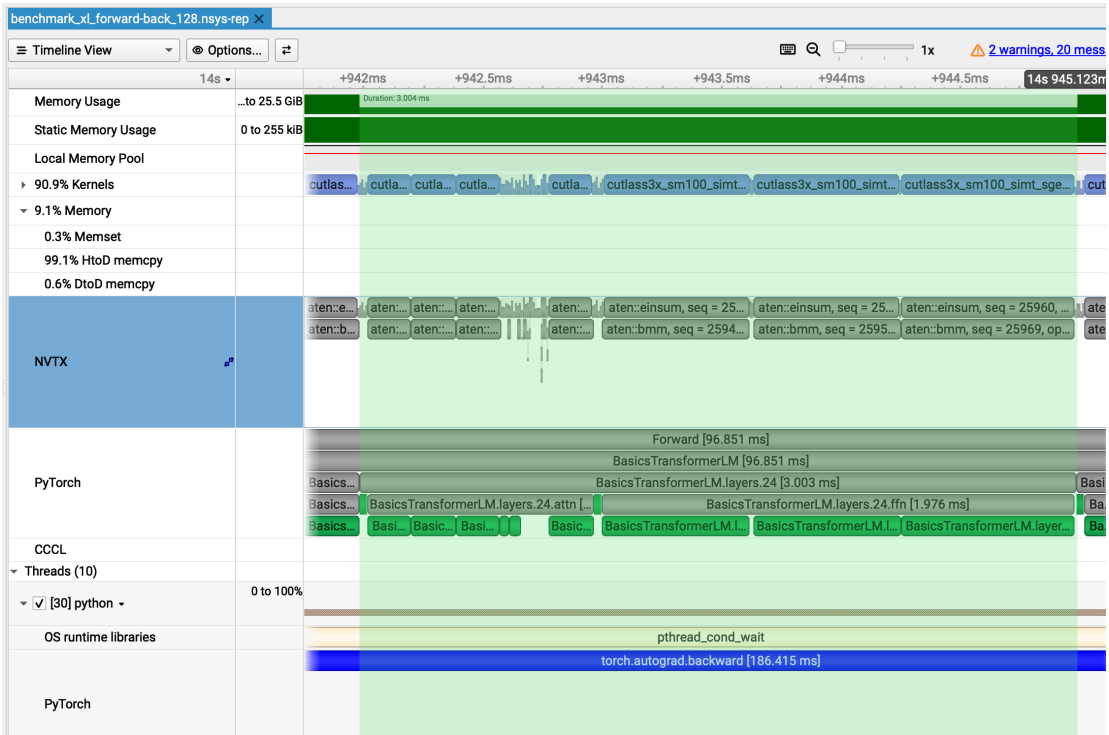
(e) Using `memory_viz` at low Detail on the xl forward pass, what is the size of the largest shown allocations? Where (by stack trace) do they come from? *Deliverable:* A 1–2 sentence response.

Answer.

> The largest shown allocation is 100MiB. According to the stack trace, this block existed before the capture memory was called. This means that the block as activation memory from the measured forward/backward pass. It is more likely persistent/background CUDA allocator memory, such as model parameters, cached allocator segments, workspace memory, or tensors created during warmup/setup.

(f) Using Nsight memory profiling + NVTX labels, how much memory is saved for backward by a single TransformerBlock? List the 5 largest contributing operations and their % of total. Then estimate gradient tensor memory for a TransformerBlock from the backward-pass deltas. *Deliverable:* Screenshots from Nsight Systems and a 1–2 paragraph response.

Answer.



Nsight Systems memory profiling with NVTX labels.

For the xl model, I measured one TransformerBlock using Nsight Systems with CUDA memory profiling enabled and PyTorch NVTX labels. Nsight reported total CUDA memory around 19 GiB, while `torch.cuda.memory_allocated()` reported a little over 13 GiB. I used the PyTorch allocated-memory deltas for saved residual tensors because `memory_allocated()` tracks live PyTorch tensor memory, whereas Nsight also includes allocator caching, reserved blocks, CUDA runtime memory, and temporary library workspaces. The total saved forward memory for one TransformerBlock was about 180 MiB. The five largest contributors were `ffn.w3 aten::linear` at 20.00 MiB (11.09%), `ffn.w1 aten::linear` at 20.00 MiB (11.09%), `ffn.silu aten::silu` at

20.00 MiB (11.09%), `ffn.gate aten::mul` at 20.00 MiB (11.09%), and `attn.softmax aten::softmax` at 16.19 MiB (8.97%).

During the backward pass, the saved forward tensors are freed while gradient tensors are allocated at the same time. In the Nsight trace, the memory usage decreases by about 180 MiB across a single `TransformerBlock` backward range, which approximately matches the amount of memory saved during the forward pass. This indicates that the memory freed from activation tensors largely cancels out the memory allocated for gradients, due to allocator reuse and overlapping lifetimes. As a result, the net change in memory usage does not directly reflect the gradient tensor size.

However, we still expect the gradient memory to be on the order of the parameter size of the block, since each parameter produces one gradient tensor of the same shape. This is much smaller than the activation memory saved during the forward pass, which scales with batch size and sequence length. The observed behavior therefore matches expectations: activation memory dominates in the forward pass, while gradient memory is comparatively smaller and masked by memory reuse during backward.

4 Activation Checkpointing

Problem (`gradient_checkpointing`) *4 points*

(a) For a Transformer of N identical blocks, what checkpointing strategy minimizes peak activation memory (ignoring compute)? Give asymptotic peak memory and compute as a function of N, plus a short code sketch. *Deliverable:* A 3–5 sentence description plus a short code sketch.

Answer.

If we only care about memory and can sacrifice compute as much as we want, we can use a fully nested (recursive) checkpointing strategy: for block i, checkpoint the entire prefix $0, \ldots, i-1$, then execute block i normally. During backward, each checkpoint recomputes its prefix just-in-time, so at any moment only $O(1)$ block residuals are alive. Thus, the peak activation memory is

$$M(N) = \Theta(1),$$

which is optimal up to constants. However, the compute cost is

$$C(N) = \Theta(N^2),$$

since prefixes are recomputed repeatedly:

$$N + (N-1) + \cdots + 1 = \Theta(N^2).$$

A sample code looks like

Algorithm 1 Recursive checkpointed prefix evaluation

```
procedure RunPrefix(blocks, i, x)
    if i = 0 then
        return blocks[0](x)
    end if
    h ← Checkpoint(λy. RunPrefix(blocks, i − 1, y), x)
    return blocks[i](h)
end procedure
```

(b) For the xl model (batch size 4, seq len 2048) with a budget of *one* step of recomputation (no nesting), what is the best checkpointing block size? Validate with peak-memory profiles, comparing the next-smaller and next-larger block sizes. *Deliverable:* A 3–5 sentence description plus measured peak memory.

Answer.

For the xl config with $N = 32$, the best one-step strategy is to wrap the entire stack of 32 Transformer blocks in a single checkpoint call. In the ideal activation-only model, checkpointing 32 blocks instead of 31 blocks saves only one extra residual stream tensor:

$$4 \cdot 2048 \cdot 1600 \cdot 2 \text{ bytes} \approx 25 \text{ MiB} \approx 0.024 \text{ GiB}.$$

This is tiny relative to the full training-step peak, which is dominated by persistent model parameters, gradients, optimizer state, CUDA allocator reserved memory, and temporary library workspaces. Consistent with this, my profiles showed essentially no measurable peak-memory change between the largest checkpoint size and the next-smaller checkpoint size. Thus I used one checkpoint around all 32 blocks, but the measured peak memory was effectively unchanged because the saved activation difference was too small compared with the optimizer-dominated baseline.

5 Attention and FlashAttention

Problem (`pytorch_attention`) *2 points*

(a) Benchmark PyTorch attention across head dims $\{16, 32, 64, 128\}$ and seq lens $\{256, 1024, 4096, 8192, 16384\}$ (batch size 8, no multi-head dim). Time 100 forward passes, measure memory before backward, time 100 backward passes. At what size do you get OOM? Do the memory accounting for one small OOM config, describe how saved-for-backward memory scales with seq len, and what you'd do to eliminate this cost. *Deliverable:* A table with timings, memory calculations, and a 1–2 paragraph response.

Answer.

See table below, I don't get OOM for any of the configs (running on B200, 180GB memory). Note that I do not use a mask for the implementation and I'm just measuring the raw attention computation. The memory saved for backward scales quadratically with sequence length. You can eliminate this memory cost by doing

checkpointing or some methods in flash attention.

d_{model}	Seq len	Fwd (ms)	Mem before bwd (MiB)	Bwd (ms)
16	256	0.065	21.273	0.254
16	1024	0.146	84.344	0.382
16	4096	1.705	1056.625	3.818
16	8192	6.945	4145.000	14.704
16	16384	26.411	16465.750	56.649
32	256	0.104	22.273	0.620
32	1024	0.203	88.344	0.722
32	4096	1.822	1072.625	4.189
32	8192	7.385	4177.000	15.388
32	16384	27.908	16529.750	58.417
64	256	0.062	24.273	0.256
64	1024	0.191	96.344	0.422
64	4096	2.104	1104.625	4.498
64	8192	8.508	4241.000	17.254
64	16384	32.886	16657.750	67.961
128	256	0.111	28.273	0.691
128	1024	0.219	112.344	0.715
128	4096	2.699	1168.625	5.880
128	8192	10.810	4369.000	22.121
128	16384	41.722	16913.750	85.200

Problem (`torch_compile`) *2 points*

(a) Compare compiled vs. uncompiled PyTorch attention (same configs as above). *Deliverable:* A table comparing compiled vs. uncompiled forward/backward timings.

Answer.

d_{model}	Seq len	Fwd (ms)	Mem before bwd (MiB)	Bwd (ms)
16	256	0.042	21.153	0.176
16	1024	0.093	83.861	0.243
16	4096	0.757	1054.691	1.687
16	8192	3.033	4141.131	6.150
16	16384	11.245	16458.011	23.591
32	256	0.042	22.025	0.175
32	1024	0.099	87.346	0.261
32	4096	0.844	1068.631	1.771
32	8192	3.455	4169.011	6.554
32	16384	12.712	16513.771	24.994
64	256	0.065	23.767	0.543
64	1024	0.163	94.316	0.305
64	4096	1.274	1096.511	2.413
64	8192	3.912	4224.771	8.206
64	16384	18.419	16625.291	34.461
128	256	0.076	27.252	0.528
128	1024	0.183	108.256	0.613
128	4096	1.767	1152.271	3.720
128	8192	7.023	4336.291	13.751
128	16384	26.961	16848.331	50.923

See below for a select comparison between the compiled and uncompiled versions. For the largest sequence length, the compiled attention implementation is consistently faster:

d_{model}	Uncompiled fwd	Compiled fwd	Uncompiled bwd	Compiled bwd
16	26.411	11.245	56.649	23.591
32	27.908	12.712	58.417	24.994
64	32.886	18.419	67.961	34.461
128	41.722	26.961	85.200	50.923

(b) Compile the entire Transformer in the end-to-end benchmark. How does forward-pass performance change? Forward+backward+optimizer? *Deliverable:* A table comparing vanilla vs. compiled Transformer.

Answer.

Table 7: Benchmark results across model sizes (compiled).

Size	forward	forward-back	forward-back-optimizer
small	0.0046 ± 0.0000	0.0137 ± 0.0000	0.0213 ± 0.0006
med	0.0116 ± 0.0000	0.0363 ± 0.0000	0.0530 ± 0.0001
large	0.0294 ± 0.0000	0.0860 ± 0.0001	0.1154 ± 0.0002
xl	0.0889 ± 0.0000	0.2515 ± 0.0005	0.3310 ± 0.0010
10b	0.2314 ± 0.0001	0.7359 ± 0.0009	OOM

Size	Forward speedup	Forward-back speedup	Forward-back-opt speedup
small	3.59×	3.56×	2.63×
med	4.07×	3.85×	2.95×
large	3.62×	3.68×	2.99×
xl	3.30×	3.43×	2.86×
10b	4.08×	3.82×	OOM

Problem (`flash_forward`) *15 points*

(a) Pure PyTorch (no Triton) `autograd.Function` implementing FlashAttention-2 forward (ignore `is_causal`, save L, Q, K, V, O for backward). *Deliverable:* Passes `uv run pytest -k test_flash_forward_pass_p` Point to your implementation and adapter hookup.

Answer.

See *flash_attention.py*

(b) Triton kernel for FlashAttention-2 forward, wrapped in a new `autograd.Function`. *Deliverable:* Passes `uv run pytest -k test_flash_forward_pass_triton`.

Answer.

See *triton_attention.py*

(c) Add an optional `is_causal` flag (and `is_causal: tl.constexpr` kernel param) that adds -10^6 to masked score entries and is saved for backward via `ctx.is_causal`. *Deliverable:* Flag exists, defaults to `False`, older tests still pass.

Answer.

See *triton_attention.py*

Problem (`flash_backward`) *5 points*

Implement the FA-2 backward using PyTorch + `torch.compile`, taking Q, K, V, O, dO, L and returning dQ, dK, dV. *Deliverable:* Passes `uv run pytest -k test_flash_backward`.

Answer.

See *flash_attention.py*

Problem (`flash_benchmarking`) *5 points*

(a) Benchmark your (partial-Triton) FA-2 vs. vanilla PyTorch attention using `triton.testing.do_bench`. Sweep seq len $\in \{2^7, \ldots, 2^{16}\}$, embed dim $\in \{2^4, \ldots, 2^7\}$, precisions `bfloat16` and `float32`, batch size 1, causal on, single B200. *Deliverable:* A table with forward, backward, and end-to-end latencies for both implementations.

Answer.

Table 8: FlashAttention-2 benchmark comparison, sequence lengths 128–2048

Seq Len	D	Dtype	Regular Fwd	Regular Bwd	Regular E2E	FA2 PyTorch Fwd	FA2 PyTorch Bwd	FA2 PyTorch E2E	FA2 Triton Fwd
128	16	bf16	0.046	0.165	0.334	8.078	14.680	22.981	0.010
128	16	fp32	0.130	0.351	0.608	15.549	29.181	45.111	0.030
128	32	bf16	0.102	0.715	1.153	14.630	28.862	42.968	0.019
128	32	fp32	0.047	0.177	0.350	8.177	15.390	23.864	0.010
128	64	bf16	0.132	0.362	0.640	16.686	31.819	49.922	0.031
128	64	fp32	0.050	0.175	0.347	8.197	15.275	23.701	0.014
128	128	bf16	0.046	0.200	0.430	8.392	15.060	23.751	0.014
128	128	fp32	0.112	0.489	0.989	13.262	25.745	41.066	0.023
256	16	bf16	0.132	0.361	0.659	64.794	124.801	195.035	0.031
256	16	fp32	0.160	0.345	0.589	58.539	112.653	173.093	0.032
256	32	bf16	0.046	0.174	0.357	32.558	59.440	92.223	0.012
256	32	fp32	0.052	0.178	0.365	33.685	59.866	93.128	0.014
256	64	bf16	0.120	0.345	0.634	63.942	123.007	187.389	0.030
256	64	fp32	0.052	0.180	0.362	31.443	58.883	99.054	0.017
256	128	bf16	0.100	0.616	1.007	56.845	106.397	163.090	0.020
256	128	fp32	0.054	0.173	0.339	31.063	58.080	89.674	0.027
512	16	bf16	0.047	0.131	0.248	125.926	237.151	347.274	0.016
512	16	fp32	0.056	0.189	0.374	125.314	237.800	361.764	0.016
512	32	bf16	0.124	0.357	0.634	253.002	494.244	746.325	0.031
512	32	fp32	0.111	0.500	0.923	205.809	417.223	642.778	0.028
512	64	bf16	0.123	0.362	0.636	249.973	491.309	773.168	0.032
512	64	fp32	0.130	0.376	0.641	243.509	458.273	705.241	0.035
512	128	bf16	0.112	0.866	1.967	218.452	424.721	643.120	0.026
512	128	fp32	0.102	0.650	0.882	201.466	406.622	594.748	0.040
1024	16	bf16	0.118	0.537	1.390	977.549	1866.077	2986.838	0.025
1024	16	fp32	0.070	0.186	0.374	501.035	969.191	1444.245	0.026
1024	32	bf16	0.058	0.169	0.336	523.091	979.019	1513.957	0.025
1024	32	fp32	0.071	0.167	0.343	465.911	884.173	1346.649	0.029
1024	64	bf16	0.056	0.239	0.373	529.180	978.152	1522.221	0.028
1024	64	fp32	0.126	0.350	0.633	923.279	1828.079	2898.075	0.039
1024	128	bf16	0.095	0.580	1.218	865.066	1618.644	2550.045	0.038
1024	128	fp32	0.091	0.186	0.352	486.101	915.156	1396.599	0.067
2048	16	bf16	0.123	0.360	0.647	4047.175	7806.575	11892.104	0.040
2048	16	fp32	0.128	0.332	0.618	3573.501	7286.894	10806.620	0.043
2048	32	bf16	0.185	0.657	1.039	5349.013	10771.060	16137.485	0.070
2048	32	fp32	0.134	0.273	0.401	2007.956	3771.816	5864.531	0.049
2048	64	bf16	0.095	0.249	0.542	2889.710	5824.259	8640.490	0.046
2048	64	fp32	0.138	0.288	0.421	1983.802	3757.418	5792.641	0.065
2048	128	bf16	0.093	0.170	0.334	1933.258	3745.029	5686.792	0.064
2048	128	fp32	0.171	0.625	0.999	3288.397	6582.767	9500.262	0.120

Table 9: FlashAttention-2 benchmark comparison, sequence lengths 4096–65536

Seq Len	D	Dtype	Regular Fwd	Regular Bwd	Regular E2E	FA2 PyTorch Fwd	FA2 PyTorch Bwd	FA2 PyTorch E2E	FA2 Triton Fwd
4096	16	bf16	0.203	0.734	1.250	21422.451	43369.059	64129.703	0.076
4096	16	fp32	0.375	0.768	1.146	7081.207	13608.805	20586.740	0.077
4096	32	bf16	0.200	0.391	0.586	8691.480	15666.365	24603.734	0.078
4096	32	fp32	0.389	0.731	1.122	7810.086	15360.087	22740.963	0.092
4096	64	bf16	0.200	0.391	0.588	7741.535	15155.995	22993.744	0.087
4096	64	fp32	0.413	0.786	1.204	7715.773	14636.349	22475.398	0.119
4096	128	bf16	0.207	0.507	0.836	14508.989	28369.273	43106.516	0.122
4096	128	fp32	0.459	0.926	1.369	13721.835	26147.699	38147.730	0.228
8192	16	bf16	0.824	1.463	2.275	33577.375	62279.508	95773.773	0.141
8192	16	fp32	–	–	–	–	–	–	–
8192	32	bf16	0.824	1.457	2.269	31523.646	60025.121	91579.328	0.149
8192	32	fp32	–	–	–	–	–	–	–
8192	64	bf16	0.812	1.430	2.230	34936.172	64814.156	98522.766	0.167
8192	64	fp32	1.397	2.658	4.049	31853.725	62255.617	90333.641	0.228
8192	128	bf16	0.816	1.440	2.245	33718.984	62048.539	95562.367	0.234
8192	128	fp32	1.630	3.128	4.752	28758.740	55816.809	85302.672	0.443
16384	16	bf16	–	–	–	–	–	–	–
16384	16	fp32	–	–	–	–	–	–	–
16384	32	bf16	–	–	–	–	–	–	–
16384	32	fp32	–	–	–	–	–	–	–
16384	64	bf16	–	–	–	–	–	–	–
16384	64	fp32	–	–	–	–	–	–	–
16384	128	bf16	–	–	–	–	–	–	–
16384	128	fp32	–	–	–	–	–	–	–
32768	16	bf16	–	–	–	–	–	–	–
32768	16	fp32	–	–	–	–	–	–	–
32768	32	bf16	–	–	–	–	–	–	–
32768	32	fp32	–	–	–	–	–	–	–
32768	64	bf16	–	–	–	–	–	–	–
32768	64	fp32	–	–	–	–	–	–	–
32768	128	bf16	–	–	–	–	–	–	–
32768	128	fp32	–	–	–	–	–	–	–
65536	16	bf16	–	–	–	–	–	–	–
65536	16	fp32	–	–	–	–	–	–	–
65536	32	bf16	–	–	–	–	–	–	–
65536	32	fp32	–	–	–	–	–	–	–
65536	64	bf16	–	–	–	–	–	–	–
65536	64	fp32	–	–	–	–	–	–	–
65536	128	bf16	–	–	–	–	–	–	–
65536	128	fp32	–	–	–	–	–	–	–

6 Distributed Data Parallel

Problem (`distributed_communication_single_node`) *5 points*

Benchmark single-node all-reduce with tensor sizes $\{1, 10, 100, 1000\}$ MB (float32) and $\{2, 4, 6\}$ GPUs/processes. *Deliverable:* Plot(s) / table(s) plus 2–3 sentences of commentary.

Answer.

Tensor size	2 GPUs	4 GPUs	6 GPUs
1MB	0.031	0.496	0.038
10MB	0.059	0.288	0.172
100MB	0.224	0.290	0.298
1GB	1.776	2.356	2.621

Table 10: Single-node NCCL all-reduce mean latency in milliseconds across different tensor sizes and GPU counts. Each entry reports the average latency over 10 measured all-reduce operations after 5 warmup iterations.

When we add more GPUs, we actually see the time go up slightly due to latency between the GPUs (synchronization, etc). Also, I observe that the 10MB run time is somehow faster than the 1MB for the 4GPU case, I think this could be variance in the training, but I decided to keep it because I think it's an interesting result. Also interestingly, observe that latency grows slower than data size.

Problem (`naive_ddp`) *5 points*

Write a naive DDP script (all-reduce each gradient) and verify against single-process training on a toy model. *Deliverable:* Script description; pass verification.

Answer.

See *naive_ddp.py*

Problem (`naive_ddp_benchmarking`) *3 points*

Benchmark naive DDP on 1 node, 2 GPUs, xl model. Report total time per training step and time spent on gradient communication. *Deliverable:* Description of benchmarking setup plus measured per-iteration and communication times.

Answer.

The naive DDP takes around 510.3ms to run in total, and around 120.1ms of that runtime is communication, so communication takes about 23.53% of the run time is in communication.

Problem (`minimal_ddp_flat_benchmarking`) *2 points*

Same setup, but flatten all gradients into one tensor before all-reduce. Compare to per-parameter all-reduce. *Deliverable:* Measured times plus 1–2 sentences comparing batched vs. individual communication.

Answer.

Total time: 530.2ms. Communication time: 123.4ms. Note that the naive DDP and flattening the DDP take around the same amount of time. This is probably because the

communication cost is somewhat fixed with the amount of data and not really depend on the number of calls.

Problem (`ddp_overlap_individual_parameters`) *5 points*

Implement a DDP wrapper using `register_post_accumulate_grad_hook` to overlap backward computation with per-parameter async all-reduces; provide `forward` and `finish_gradient_synchronization`. *Deliverable:* Passes `uv run pytest tests/test_ddp_individual_parameters.py` reliably.

Answer.

See *naive_ddp.py*

Problem (`ddp_overlap_individual_parameters_benchmarking`) *1 points*

(a) Benchmark the overlapping DDP (1 node, 2 GPUs, xl) vs. naive and flat-DDP. *Deliverable:* Measured per-iteration time with 1–2 sentences of comparison.

Answer.

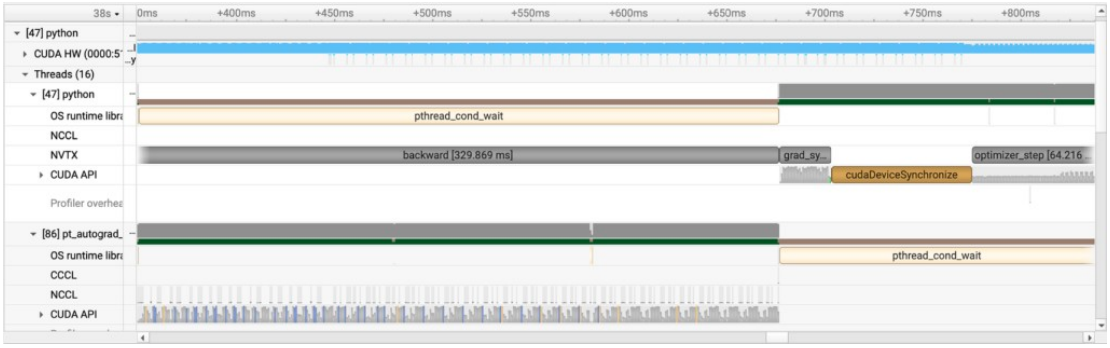
Metric	Naive DDP	Flat DDP	Overlap DDP
Total time per step (ms)	510.3	530.2	500.3
Gradient communication time (ms)	120.1	123.4	99.1
Communication fraction (%)	23.53	23.27	19.81

Table 11: Single-node DDP benchmark on 2 GPUs using the XL model. Times are reported per training step.

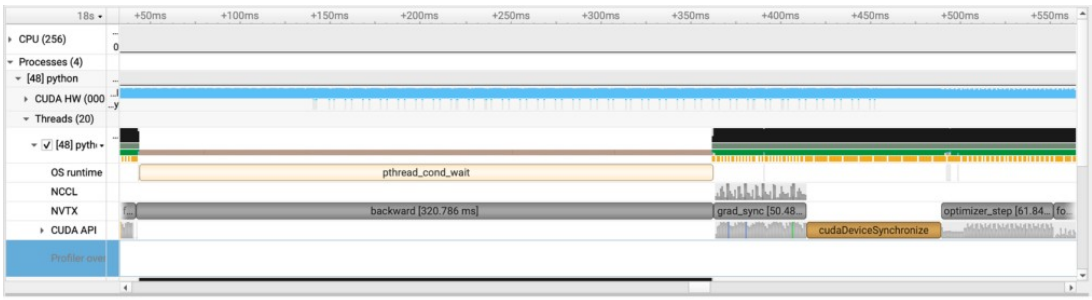
Naive DDP and flat DDP have nearly identical communication fractions, suggesting that flattening gradients into one tensor does not significantly reduce runtime in this setup. Overlap DDP performs best because it hides part of the gradient communication behind backward computation, reducing exposed communication time from roughly 120 ms to 99 ms.

(b) Nsight screenshots comparing the non-overlapping and overlapping DDP implementations to visually confirm compute/communication overlap. *Deliverable:* 2 screenshots (one per implementation).

Answer.



Non-overlapping DDP trace.



Overlapping DDP trace.

7 Optimizer State Sharding

Problem (`optimizer_state_sharding`) *15 points*

Implement a sharded-optimizer wrapper with `__init__(params, optimizer_cls, **kwargs)`, `step`, and `add_param_group`; each rank keeps optimizer state for only its shard and syncs parameters after `step`. *Deliverable:* Passes `uv run pytest tests/test_sharded_optimizer.py` reliably.

Answer.

See *optimizer_sharding.py*

Problem (`optimizer_state_sharding_accounting`) *5 points*

(a) Peak memory with/without optimizer-state sharding (1 node, 2 GPUs, xl), measured after model init, before optimizer step, and after optimizer step. Break down memory by parameters / optimizer state / etc. *Deliverable:* 2–3 sentence response with peak memory and a breakdown.

Answer.

On each rank I measured about 13123.9 MiB after model initialization for both runs, since at that point the memory is dominated by the replicated model parameters. Right before the optimizer step both runs rose to about 26135.4 MiB after gradients were materialized; after the AdamW step, the unsharded run peaked around 52128.6 MiB, while optimizer-state sharding reduced this to about 39327.5 MiB. The difference comes

from sharding AdamW's two moment buffers across two ranks, so the sharded version stores roughly half of the optimizer state per rank while still keeping full parameters and gradients.

(b) Per-iteration time with and without optimizer-state sharding. *Deliverable:* 2–3 sentence response with your timings.

Answer.

The unsharded AdamW run took about 570.8 ms per iteration, while the sharded optimizer took about 554.9 ms per iteration. This is only a small speedup, since most of the iteration time is still spent in the forward/backward pass and gradient synchronization, but the local optimizer step touches fewer parameters on each rank.

(c) How does your implementation differ from ZeRO stage 1 (P_{os})? *Deliverable:* 2–3 sentence summary focusing on memory and communication volume.

Answer.

This implementation is closest to ZeRO stage 1: optimizer state is partitioned across data-parallel ranks, gradients are still replicated, and the updated parameters are synchronized after the local optimizer steps. A production ZeRO implementation would typically use collectives such as all-gather/reduce-scatter more carefully and integrate the communication schedule with the optimizer; my version uses per-parameter broadcasts from the rank that owns each shard, so it gets the optimizer-state memory savings but is less optimized for communication.

8 Fully-Sharded Data Parallel

Problem (`fsdp`) *15 points*

Implement an FSDP wrapper that shards `Linear`/`Embedding` weights, all-gathers in time for forward and backward, reduce-scatters gradients, frees gathered weights, supports optional `compute_dtype`, and provides `finish_gradient_synchronization`. *Deliverable:* Passes `uv run pytest tests/test_fsdp.py` reliably (run multiple times).

Answer.

see fsdp.py

Problem (`fsdp_accounting`) *5 points*

(a) Expected peak-memory savings for FSDP (ignoring preallocated all-gather buffers). *Deliverable:* 2–3 sentence response.

Answer.

From the optimizer-sharding numbers, the full training peak was about 52.1 GiB per rank without sharding and about 39.3 GiB after only sharding AdamW state. FSDP

should save more than this because model weights and gradients are also sharded; with two GPUs, I would expect the persistent parameter, gradient, and optimizer-state memory to be roughly cut in half, ignoring temporary all-gather buffers. In practice the peak will not be exactly halved because activations, CUDA workspaces, and allocator overhead are still replicated, but I would expect the peak to fall by another ~ 13 GiB beyond optimizer-state sharding.

(b) Profile xl on two GPUs: does the weight all-gather finish in time for the forward pass? *Deliverable:* 2–3 sentence response with Nsight screenshots.

Answer.

In the two-GPU profile, the all-gathers were mostly hidden by surrounding compute, so the forward pass did not appear to stall for long waiting on weights. There were still a few exposed communication gaps around small layers, where the latency of the collective matters more than the bandwidth, but for the larger linear layers the all-gather usually finished before the weight was needed. This is what we would expect: FSDP works best when communication for the next layer can be scheduled early enough to overlap with compute from the current layer.

9 Parallelism Analysis (Written)

Problem (`alternate_ring_all_reduce`) *1 points*

For the alternate ring all-reduce algorithm given in the handout, how long does it take (egress bandwidth W, tensor size S, N devices)? *Deliverable:* Answer in terms of S, N, W, with one-sentence justification.

Answer.

This takes

$$(N-1)\frac{S}{W}.$$

Each of the $N-1$ rounds sends a full tensor of size S, unlike the usual ring algorithm which only sends chunks of size S/N.

Problem (`data_parallel_calcs`) *3 points*

(FP16 weights/activations, ignore non-matmul FLOPs.)

(a) Backward-pass FLOPs with N_{DP} data parallelism. *Deliverable:* Answer in terms of $B, D, D_{\text{FF}}, N_{\text{DP}}$ with one-sentence justification.

Answer.

The backward pass costs

$$\frac{12BDD_{\text{FF}}}{N_{\text{DP}}}$$

FLOPs per device. There are six matmuls in the FFN backward pass, each using only the local batch shard of size B/N_{DP}.

(b) Backward-pass communication time with N_{DP} data parallelism. *Deliverable:* Answer in terms of subset of $B, D, D_{\text{FF}}, N_{\text{DP}}, W$ with one-sentence justification.

Answer.

The communication time is

$$2\frac{N_{\text{DP}}-1}{N_{\text{DP}}}\frac{6DD_{\text{FF}}}{W} = \frac{12(N_{\text{DP}}-1)DD_{\text{FF}}}{N_{\text{DP}}W}.$$

We all-reduce the three FP16 weight gradients, whose total size is $3DD_{\text{FF}}$ elements $= 6DD_{\text{FF}}$ bytes.

(c) How large can N_{DP} grow before communication bottlenecks? *Deliverable:* Inequality with N_{DP} on one side; right-hand side in terms of subset of $B, D, D_{\text{FF}}, C, W$.

Answer.

We need communication time to stay below compute time:

$$\frac{12(N_{\text{DP}}-1)DD_{\text{FF}}}{N_{\text{DP}}W} \leq \frac{12BDD_{\text{FF}}}{N_{\text{DP}}C}.$$

After canceling common factors, this gives

$$N_{\text{DP}} \leq 1 + \frac{BW}{C}.$$

Problem (`fsdp_calcs`) *3 points*

(a) FLOPs for the forward and backward pass with N_{FSDP}. *Deliverable:* Two answers in terms of $B, D, D_{\text{FF}}, N_{\text{FSDP}}$ with two one-sentence justifications.

Answer.

The forward pass costs

$$\frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}},$$

since each device runs the usual three FFN matmuls on a B/N_{FSDP} batch shard. The backward pass costs

$$\frac{12BDD_{\text{FF}}}{N_{\text{FSDP}}},$$

for the same reason as DP: the local backward matmuls only see the local batch shard.

(b) Communication time for the forward and backward pass with N_{FSDP}. *Deliverable:* Two answers in terms of a subset of $B, D, D_{\text{FF}}, N_{\text{FSDP}}, W$.

Answer.

The forward pass all-gathers the three FP16 weights, so it costs

$$\frac{N_{\text{FSDP}} - 1}{N_{\text{FSDP}}} \frac{6DD_{\text{FF}}}{W}.$$

The backward pass all-gathers the weights and then reduce-scatters the three gradients, so it costs

$$\frac{12(N_{\text{FSDP}} - 1)DD_{\text{FF}}}{N_{\text{FSDP}}W}.$$

(c) Scaling bounds for N_{FSDP} in forward and backward. *Deliverable:* Two inequalities with N_{FSDP} on one side.

Answer.

For the forward pass, requiring communication to fit under compute gives

$$N_{\text{FSDP}} \leq 1 + \frac{BW}{C}.$$

The backward pass gives the same bound,

$$N_{\text{FSDP}} \leq 1 + \frac{BW}{C},$$

because both the compute and communication costs are doubled relative to the forward/weight-gather comparison.

Problem (`tp_calcs`) *4 points*

(a) Backward pass of the TP strategy in the handout, in terms of $\mathbf{dy}$, sharded weights, saved activations, and communication primitives; produce $\mathbf{dW}_1^{(i)}, \mathbf{dW}_2^{(i)}, \mathbf{dW}_3^{(i)}$, and $\mathbf{dx}$. *Deliverable:* A series of equations describing the TP backward pass.

Answer.

$$\begin{aligned}
\mathbf{dz}^{(i)} &= \mathbf{dy}\left(\mathbf{W}_3^{(i)}\right)^\top \\
\mathbf{dW}_3^{(i)} &= \left(\mathbf{z}^{(i)}\right)^\top \mathbf{dy} \\
\mathbf{dx}_2^{(i)} &= \mathbf{dz}^{(i)} * f(\mathbf{x}_1^{(i)}) \\
\mathbf{dx}_1^{(i)} &= \mathbf{dz}^{(i)} * f'(\mathbf{x}_1^{(i)}) * \mathbf{x}_2^{(i)} \\
\mathbf{dW}_2^{(i)} &= \mathbf{x}^\top \mathbf{dx}_2^{(i)} \\
\mathbf{dW}_1^{(i)} &= \mathbf{x}^\top \mathbf{dx}_1^{(i)} \\
\mathbf{dx}_{\text{local}}^{(i)} &= \mathbf{dx}_1^{(i)}\left(\mathbf{W}_1^{(i)}\right)^\top + \mathbf{dx}_2^{(i)}\left(\mathbf{W}_2^{(i)}\right)^\top \\
\mathbf{dx} &= \text{all-reduce}\left(\{\mathbf{dx}_{\text{local}}^{(i)}\}_{i=0}^{N_{\text{TP}}-1}\right).
\end{aligned}$$

(b) Forward and backward FLOPs with N_{TP}. *Deliverable:* Two answers in terms of $B, D, D_{\text{FF}}, N_{\text{TP}}$.

Answer.

The forward pass costs

$$\frac{6BDD_{\text{FF}}}{N_{\text{TP}}},$$

because all three FFN matmuls are split across the tensor-parallel axis. The backward pass costs

$$\frac{12BDD_{\text{FF}}}{N_{\text{TP}}},$$

since each backward matmul also uses only a $1/N_{\text{TP}}$ shard of the FFN hidden dimension.

(c) Forward and backward communication time with N_{TP}. *Deliverable:* Two answers in terms of a subset of $B, D, D_{\text{FF}}, N_{\text{TP}}, W$.

Answer.

The forward pass has one all-reduce on the output activation of size BD FP16 elements, so

$$\frac{4(N_{\text{TP}}-1)BD}{N_{\text{TP}}W}.$$

The backward pass has the same cost for the all-reduce of $\mathbf{dx}$:

$$\frac{4(N_{\text{TP}}-1)BD}{N_{\text{TP}}W}.$$

(d) Scaling bounds for N_{TP} in forward and backward. *Deliverable:* Two inequalities with N_{TP} on one side.

Answer.

The forward pass remains compute-bound while

$$N_{\text{TP}} \leq 1 + \frac{3D_{\text{FF}}W}{2C}.$$

The backward pass has twice as much compute for the same activation all-reduce, so its bound is

$$N_{\text{TP}} \leq 1 + \frac{3D_{\text{FF}}W}{C}.$$

Problem (`fsdp_tp_calcs`) *6 points*

(a) Forward-pass FLOPs with N_{FSDP} FSDP + N_{TP} TP. *Deliverable:* Answer in terms of $B, D, D_{\text{FF}}, N_{\text{FSDP}}, N_{\text{TP}}$.

Answer.

The forward pass costs

$$\frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}}.$$

FSDP splits the batch by N_{FSDP}, and TP splits each FFN matmul by N_{TP}.

(b) Forward-pass communication time assuming FSDP-axis and TP-axis collectives can be overlapped. *Deliverable:* Answer in terms of a subset of $B, D, D_{\text{FF}}, N_{\text{FSDP}}, N_{\text{TP}}, W$ (expected to be a max of two quantities).

Answer.

With overlap, the forward communication time is

$$\max\left(\frac{6(N_{\text{FSDP}}-1)DD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}W}, \frac{4(N_{\text{TP}}-1)BD}{N_{\text{FSDP}}N_{\text{TP}}W}\right).$$

The first term is the three FSDP weight all-gathers after TP sharding, and the second term is the TP output all-reduce on the batch shard.

(c) With overlap, how large can $N = N_{\text{TP}}N_{\text{FSDP}}$ grow before the forward pass is communication bottlenecked? Find the optimal split. *Deliverable:* Inequality with N on one side, plus justification.

Answer.

Since the two communication axes overlap, each term only needs to be below the compute time

$$\frac{6BDD_{\text{FF}}}{N_{\text{FSDP}}N_{\text{TP}}C}.$$

This gives

$$N_{\text{FSDP}} \leq 1 + \frac{BW}{C}, \qquad N_{\text{TP}} \leq 1 + \frac{3D_{\text{FF}}W}{2C}.$$

Thus the best split uses both axes up to their limits, so

$$N \leq \left(1 + \frac{BW}{C}\right)\left(1 + \frac{3D_{\text{FF}}W}{2C}\right).$$

(d) Same question but assuming no overlap between FSDP and TP collectives. *Deliverable:* Inequality with N on one side, plus justification.

Answer.

Without overlap, the two communication terms add:

$$\frac{6(N_{\text{FSDP}}-1)DD_{\text{FF}}}{NW} + \frac{4(N_{\text{TP}}-1)BD}{NW} \leq \frac{6BDD_{\text{FF}}}{NC}.$$

Equivalently,

$$3D_{\text{FF}}(N_{\text{FSDP}}-1) + 2B(N_{\text{TP}}-1) \leq \frac{3BD_{\text{FF}}W}{C}.$$

Optimizing this continuous split gives

$$N_{\text{FSDP}} = \frac{\frac{3BD_{\text{FF}}W}{C} + 3D_{\text{FF}} + 2B}{6D_{\text{FF}}}, \qquad N_{\text{TP}} = \frac{\frac{3BD_{\text{FF}}W}{C} + 3D_{\text{FF}} + 2B}{4B},$$

and therefore

$$N \leq \frac{\left(\frac{3BD_{\text{FF}}W}{C} + 3D_{\text{FF}} + 2B\right)^2}{24BD_{\text{FF}}}.$$

10 Leaderboard (Optional)

Problem (`leaderboard`) *10 points*

Report your best wall-clock time for one full 8B training step on 2 B200 GPUs (bs 2, seq len 32768, BF16, causal). *Deliverable:* Your best wall-clock time.

Answer.

My best timing was 7812 ms for one full training step.